Supplementary Material for: SEER: Partition Refinement for Batch SQL Equivalence Checking

Junjie Yao
Software Engineering Institute, East China Normal University, Shanghai, China
junjie.yao@sei.ecnu.edu.cn

Each section supports one pointer of the main paper: Section I proves the partition invariants of Section IV-A, Section II the bound on labels and splits of Section IV-C, Section III gives the reproduction, counting rules and implementation of Section V-A, and Section IV the per-benchmark counts, including the mean and standard deviation over the five orders, behind Sections V-B to V-D. Notation follows the main paper: Σ is the schema with its constraints, Δ the value domain of bounded verification, bounded@k a label verified on all databases with at most k rows per table over Δ, and τ_M the block query of a class C with pending members M (Section IV-B).

I. Partition Invariants (Section IV-A)

Proposition S1 (Anytime partition)**.** *After every arrival and every installation, (I1) two queries in different classes return different keys on some admitted $D \in \mathcal{W}$, which satisfies Σ and replays in d; and (I2) every live edge joins two queries of one class and its verdict holds on every legal database within its bound and value domain. Hence, if every member ends with a proved or bounded edge and k is the smallest bound, equivalent queries share a class and queries of one class agree on all legal databases of at most k rows per table over Δ.*

Proof. (I1) Classes are the blocks of equal signatures over the active columns. An arrival adds a query to the block of its signature or creates a block, and an installation appends one key to every signature, so two queries whose signatures differed still differ. Coarsening or deactivating a column only merges classes, and the remaining active columns are admitted databases, each of which satisfies Σ and replayed in d at admission. (I2) An edge is added only between a member and its representative, which share a signature at that point. On a later database, a verdict that holds makes the member return the representative's key, so the two stay in one class; a different key revokes the path, and the edge is no longer live. A representative change links the new representative to the old one by a proved or bounded edge, so every live path stays within one class. For the consequence, equivalent queries return equal keys on every admitted database and therefore never split, by (I1). Queries of one class are joined to the representative by live paths, and by (I2) and the transitivity of $\equiv$ and $\sim_k$ they agree on every legal database of at most k rows per table over Δ. □

II. Proof of the Bound on Labels and Splits (Section IV-C)

Consider a run of n queries ending with m classes and s separating installations. A decision is a first member label, one installed database that splits a class, or one retry on an undecided member in a later decision pass; ρ counts the retries. A block can supply several member labels, while one database counts once regardless of how many members it separates. Unsuccessful engine calls, validity queries, bound deepening and label strengthening consume runtime and are counted in Section IV-C of the main paper.

Lemma S1 (Separating installations)**.** *If no signature column is coarsened or deactivated, a run installs at most $m-1$ databases that split a class.*

Proof. With no column coarsened or deactivated, classes can split or be created by arrivals, but never merge. The first query creates one class. An installed database either splits at least one class, which increases the number of classes by at least one, or it splits nothing and is not retained. The run ends with m classes, so at most $m-1$ installations split. □

Proposition (Labels and splits). *Consider a run of n queries that ends with m classes. If no signature column is coarsened or deactivated, at most $m-1$ installed databases split a class. If, in addition, every label is proved, or is bounded@k with Δ unchanged after its assignment and every later installed database within k rows per table over Δ, the run makes at most $n-1+\rho$ decisions.*

Proof. The first statement is Lemma S1. A proved label holds on every database. A bounded@k label holds on every database with at most k rows per table over Δ; under the second condition every later installed database is of this kind, so the labeled member's replay returns its representative's key and, by Proposition S1, the member stays in its representative's class through every split. A representative change within the class keeps the member's label valid under the same condition. The first labels therefore form a forest within the final m classes, with at most $n-m$ edges. A block adds one edge per newly labeled member, and retries on undecided members add ρ

TABLE I
PUBLISHED AND REPRODUCED RESULTS OF THE COMPARED TOOLS ON THEIR OWN INPUTS, WITH THE OFFICIAL CODE

Tool	Input	Measure	Published	Reproduced
SQLWitness	Spider test, 829 pairs	refuted	194.9 ± 1.2	195
SQLWitness	BIRD test, 1,226 pairs	refuted	628.3 ± 3.1	625
XDataN	XData-BM mutants	killed	382	370
SQLSolver	its 400 pairs	proved	384	387

decisions. Together with at most $m-1$ separating installations, the run makes at most $(n-m)+(m-1)+\rho = n-1+\rho$ decisions. □

III. SETUP DETAILS (SECTION V-A)

a) Workloads and limits: LeetCode also forms all within-problem pairs, over which SQLSolver's proofs are counted; the SQLSolver set combines Calcite, Spark, TPC-C and TPC-H pairs. LiveSQLBench uses the base-lite databases in SQLite. SQLWitness runs with 60 seconds per pair, ParSEval release v1.0.1 with 360 seconds per pair, and XDataN with 120 seconds of solver time per generated dataset; SQLSolver runs through the released JAR's batch entry. A pairwise tool receives, in one run of its official driver, every pair of distinct queries that the initial random databases leave together. The pair list is fixed when the driver starts, so a pair that a database found later in the run separates is still checked; SEER updates its partition after each admitted database, and the databases a tool finds are admitted into the same pool as SEER's. Both sides use in-process SQLite 3.37.2, PostgreSQL on XData-BM and MySQL on Calcite and Literature, and the workload budget charges every SEER step. Queries that the DBMS or the SQLWitness loader rejects have no signature and count as unsupported for SEER and the tools alike. SQLDriller returns flagged questions, not a partition, so it is not compared.

b) Platform and pairing: SEER and the compared tool run adjacent on the same core, except on LeetCode and XData-BM, where the compared tool's runs are earlier runs on the same machine, core and budget. Runs are pinned to one physical core, with its hyperthread sibling idle, of a server with two AMD EPYC 9654 processors (192 cores) and 1 TB of memory under Ubuntu 22.04, CPUs only and with a memory limit per worker. On LeetCode the ratios leave out the problems whose input SQLite rejects and the largest problems, on which SQLWitness finishes no order while SEER finishes every one.

c) Reproduction: Each tool first reproduces its published result from its official code; Table I compares the two on the published inputs, and the main paper reports the reproduced runs on our inputs. The kills that XDataN reports on XData-BM include seven duplicate rows of its mutant file, which we drop, and not every reproduced kill comes with a database that passes our validation (Table III, note b). The released SQLSolver returns *not equivalent* on four Spark pairs with ORDER BY and LIMIT or ROW_NUMBER that the paper reports as equivalent; we keep its native verdicts.

d) Implementation: SEER is written in about 10k lines of Python on Z3 4.13.4 and sqlglot 30.21. It reuses Polygon's bounded encoding, model decoding and database checks and replaces Polygon's group entry with the block query of Section IV-B. The SQLWitness engine runs as one persistent worker per workload that receives the hits of the arrival pass in microbatches, and the SQLSolver release sits behind a syntactic check of its profile.

e) Measurement: Separations count the separated pairs (killed mutants on XData-BM) whose database satisfies Σ, loads into the target DBMS and returns different keys for both queries there; the same check is applied to the databases of the compared tools. *Label strength* splits the queries of a workload by the strongest label each receives against the reference: refuted, proved, bounded-equivalent or undecided. The *request ratio* compares a tool's requested query pairs with SEER's member–representative requests at arrival and in verification, unsuccessful ones included. A block with b newly requested members counts b requests. Repeated block queries and bound deepening in the same class context add none; processing a class again after a split can add new ones. Solver calls, block and validity queries included, are counted separately. The *wall-clock ratio* divides the compared tool's time by SEER's on the same workload and core; timing covers encoding, pool execution, solver calls, database generation and admission, from the first query read to the partition written. On LeetCode SQLWitness keeps its native batch timing, while SEER's time includes admission and bounded verification.

f) Statistics and censoring: Ratios are geometric means over workloads with a bootstrap 95% confidence interval, together with the share of workloads on which SEER finishes first. Each configuration is repeated with five arrival orders drawn from seeds 0 to 4, the only random input. A run that reaches its budget is censored and keeps the separations found before the budget; ratios leave censored runs out and keep the orders of a workload that both tools finish. The effectiveness columns therefore count every tool's validated separations on its accepted input, while the efficiency columns use only paired completed runs; the completion counts in Section V show where this timing subset omits difficult workloads. The efficiency analysis uses 200 pre-sampled workloads each of Spider, BIRD and the BIRD pool, stratified with a fixed seed, and all of LiveSQLBench.

IV. MEASUREMENTS BEHIND THE QUALITATIVE STATEMENTS OF SECTION V

a) Workloads: LeetCode holds 24,050 queries in 56 problems, which give 23,994 submission–reference pairs, 20,719 of them in the 53 problems whose input SQLite accepts, and 6,766,278 within-problem pairs; the SQLWitness paper reports on 22,778 submission–reference pairs under MySQL. LiveSQLBench samples 32 queries per task at temperature 0.8. On the largest LeetCode problem (790 executable submissions, 311,655 pairs, order 0), ten pool databases place the queries in 50 classes and separate 256,541 pairs by the last arrival,

after 728 refuter requests; the run ends with 54 classes, 13 pool databases, 1,482 requests and 169 solver calls.

b) Separations: Over five orders SEER separates $31{,}722 \pm 117$ pairs against $31{,}839 \pm 51$ for SQLWitness on the BIRD pool and $31{,}268 \pm 257$ against $31{,}464 \pm 281$ on LiveSQLBench, and kills 374.6 ± 6.9 unique XData-BM mutants. On LeetCode SEER separates 9,432 of the 20,719 submission–reference pairs that SQLite accepts. SQLWitness and ParSEval kill 97 and 14 of the 392 XData-BM mutants they accept; XDataN separates 482 LeetCode pairs where SEER separates 5,308. On BIRD, 130 of ParSEval's 723 native refutations fail admission. The index separates 95% of the LeetCode pairs with a mean pool of 7.8 databases; the bounded verifier supplies 58% of the XData-BM separations with a mean pool of 5.5 databases. Figure 8(a) counts 14,958,821 separated within-workload pairs on LeetCode over five orders. On LeetCode 19% of the block queries run at $k \geq 8$ and the deepest unsatisfiable ones reach $k = 256$. Of the within-problem LeetCode pairs SEER refutes 45%, labels 24% proved or bounded and leaves 31% undecided; SQLSolver proves 37 to 612 of them per order.

c) Efficiency: On LeetCode the ratios cover the 45 of 56 problems where both tools finish an order; on eight problems of 576 to 919 submissions SQLWitness finishes no order. SEER's speedup grows from $8.4\times$ below 300 submissions to $38.3\times$ above 600 (17, 20 and 8 problems per bin). SEER is slower on four of the 45 problems, all of at most 181 submissions. Above 600 submissions SQLWitness completes 32 of 70 runs and SEER all 70; on one problem SQLWitness reaches the budget on all five orders, a ratio between 135 and 159. On BIRD-pool questions of at most 55 pairs SEER spends 95% of its time bounding pairs neither tool separates; both tools finish 199 of the 200 sampled questions. SEER's peak memory is within 3% of the compared driver's on the pair benchmarks and XData-BM and 13% higher on Literature. Requests per $n - 1$ have a median of 1.83 on LeetCode, 1.65 on LiveSQLBench and 1.67 on XData-BM, and no workload exceeds 3.0.

d) Ablation and discussion: Deciding member pairs directly leaves 35.6 instead of 79.8 XData-BM pairs undecided with $1.9\times$ the requests. Without pool reuse runs are 20% faster and separate 181 fewer XData-BM pairs. Each objective-extended model separates 3.9 further representative pairs on LeetCode and 0.22 on XData-BM, where the objective costs 22% more solver calls. A pool database separates 82 of the 200 Calcite pairs before any engine call. The class-context ratio $12.1\times$ has interval $[8.6, 17.1]$ over 44 problems; the previous version issues 35 times fewer requests than SQLWitness and runs 1.7 times faster, and on four problems of 167 to 771 submissions it reaches the budget on most orders while SEER needs 2 to 23 minutes against budgets of 2.8 to 12.8 hours. SEER makes a median of 73 solver calls per LeetCode run. One SEER request takes as long as two to three SQLWitness requests in every size bin, and the decision pass takes 82% to 94% of SEER's time. Over 265 LeetCode runs, 77% of the 14,746 block queries are unsatisfiable, 2.5% yield a pool database and 21% end unknown. Across five orders a LeetCode problem's requests vary by 0.8% at the median (at most 57%) and its time by a median factor of 1.32.

AI Acknowledgment

The text of all sections was drafted and revised with Claude (Anthropic, through Claude Code) and OpenAI Codex from outlines and instructions given by the authors, who specified the content, checked every statement against the code and the measurements, and edited the final wording. The same tools wrote, from the authors' designs, the TikZ and matplotlib sources of the figures, the implementation of the method and the baselines' ports, and the scripts that ran the experiments and produced the reported numbers. The problem formulation, the algorithms, the experimental design and the interpretation of the results are the authors'. The LLM-generated query candidates that form the evaluated workloads are experimental inputs described in Section V-A, not AI-generated content of this paper.